

Frontier Atlas — System Architecture

Asynchronous AI ecosystem data-ingestion, enrichment, validation and export pipeline

1. Architecture Overview

Layer	Components / Responsibility
Public Sources	YC company directory, AI tools directory, arXiv, GitHub, public job APIs and technology-news RSS feeds.
Ingestion	aiohttp for APIs/RSS; Playwright for dynamic web pages; concurrency limits, retries and rate-limit handling.
Processing	Parsing, date normalization, AI relevance filtering, article-text extraction, deduplication and GitHub enrichment.
LLM Orchestration	Gemini → Groq → DeepSeek fallback chain; 429 retry with exponential backoff + jitter; 413-aware chunking.
Quality & Entity	Pydantic/domain validation, required-field checks, URL checks, freshness checks and deterministic canonicalization.
Storage & Delivery	JSON datasets → Frontier_Atlas.xlsx → Google Sheets with six required tabs.

2. Data Flow

Source → Async Scraper → Structured Records → Enrichment / LLM → Validation → Entity Resolution → JSON Storage → Spreadsheet Export

3. Component Design

Scrapers	Collect source data asynchronously. aiohttp handles APIs/RSS and Playwright handles dynamic pages.
Pipelines	Coordinate collection, enrichment, validation and persistence for each dataset.
LLM Layer	Common provider interface with fallback, retries and large-input chunking.
Entity Layer	Domain models plus deterministic raw-name to canonical-name resolution.
Storage	JSON persistence and consolidated six-tab spreadsheet export.
Quality Validator	Checks counts, required fields, URLs, duplicates, timestamps and 24-hour freshness.

4. Reliability & Error Handling

429: Rate-limited requests are retried using exponential backoff and jitter; failed providers proceed through the configured fallback chain.

413: Oversized LLM prompts are split at natural text boundaries into smaller chunks and processed independently.

Source failures: Individual source failures are logged and treated as non-fatal where possible so other sources can continue.

GitHub quota: GitHub enrichment can be skipped when the public API limit is reached while preserving the core paper record.

5. Data Quality & Traceability

Check	Implementation
Record counts	Minimum 1,000 for Startups, Products and Research Papers.
Required fields	Dataset-specific names, titles, URLs and timestamps are validated.
Duplicates	Source URLs, arXiv IDs and raw entity names are checked for duplicates.
Freshness	Jobs and News are validated against a rolling 24-hour UTC window.
Entity resolution	Known name variations are deterministically normalized and logged.
Traceability	Source URLs are retained with records for provenance.

6. Current Validated Output

The automated quality gate passed with zero errors and zero warnings. Current output: 1,000 Startups, 1,000 Products, 1,000 Research Papers, 17 Jobs, 5 News records and 1,000 Entity Mapping records.

7. Scalability to 500k+ Records

The current modules separate ingestion, processing, validation and storage so the local JSON layer can evolve into distributed infrastructure without changing the core domain logic.

Current design	Production-scale evolution
Async scrapers	Queue-based distributed worker pool
Local JSON	Object storage for raw data + relational database for structured entities
Single-process pipelines	Horizontally scaled workers and partitioned workloads
Local deduplication	Central identity keys, indexes and idempotent upserts
Spreadsheet delivery	Batch exports / analytics warehouse for large datasets

8. Production-Scale Reference Design

Sources → Queue → Async Workers → Raw Object Storage → Validation / Enrichment → Entity Resolution → Database + Vector / Graph Stores → Analytics / Export

Key principles:

- Partition ingestion by source and time window.
- Use queues to absorb bursts and isolate failures.
- Retain raw payloads for auditability and replay.
- Make processing idempotent using stable source/entity keys.
- Prefer incremental updates over full rebuilds.
- Track freshness and provenance as first-class metadata.

9. Security & Operations

API keys are loaded from environment variables rather than embedded in source code. Failure handling is logged without exposing credentials. External-source failures are isolated so the pipeline can degrade gracefully.

10. Submission Artifacts

The repository contains the source code, tests, README and generated datasets. The consolidated Frontier_Atlas.xlsx contains the six required tabs: Startups, Products, Research Papers, Jobs, News and Entity Mapping Log.

Frontier Atlas — Architecture Document